```json
{
"name": "WF2_Booking_Orchestrator_v4",
"nodes": [
{
"parameters": {
"path": "booking-orchestrator-v4",
"httpMethod": "POST",
"responseMode": "lastNode",
"options": {}
},
"name": "Webhook",
"type": "n8n-nodes-base.webhook",
"typeVersion": 2.1,
"position": [0, 0],
"webhookId": "booking-orchestrator-v4"
},
{
"parameters": {
"jsCode": "// Validate Input v4.0\nconst body = $input.first().json.body || $input.first().json;\nconst required = ['provider_id', 'service_id', 'start_time'];\nfor (const field of required) {\n  if (!body[field]) {\n    return [{ json: { success: false, error_code: 'MISSING_FIELD', error_message: `Missing: ${field}`, data: null, _meta: { source: 'WF2_v4', timestamp: new Date().toISOString() } }}];\n  }\n}\nconst provider_id = parseInt(body.provider_id, 10);\nconst service_id = parseInt(body.service_id, 10);\nif (isNaN(provider_id) || isNaN(service_id)) {\n  return [{ json: { success: false, error_code: 'INVALID_TYPE', error_message: 'provider_id/service_id must be numeric', data: null, _meta: { source: 'WF2_v4', timestamp: new Date().toISOString() } }}];\n}\nconst duration = parseInt(body.duration_minutes || 60, 10);\nconst startDate = new Date(body.start_time);\nconst endDate = new Date(startDate.getTime() + duration * 60000);\nconst cleanTime = String(body.start_time).replace(/[^0-9]/g, '');\nconst idempotency_key = `booking_${provider_id}_${service_id}_${cleanTime}_${body.customer_id || body.chat_id || 'anon'}`.substring(0, 255);\nreturn [{ json: { ctx: { provider_id, service_id, start_time: body.start_time, end_time: endDate.toISOString(), duration_minutes: duration, user_id: parseInt(body.user_id || 0, 10), customer_id: body.customer_id || null, chat_id: body.chat_id || null, event_title: body.event_title || 'Appointment', idempotency_key, _meta: { source: 'WF2_Booking_Orchestrator_v4', version: '4.0.0-internal', timestamp: new Date().toISOString() } } }}];"
},
"name": "Validate Input",
"type": "n8n-nodes-base.code",
"typeVersion": 2,
"position": [250, 0]
},
{
"parameters": {
"operation": "executeQuery",
"query": "SELECT id as booking_id, status, gcal_event_id FROM bookings WHERE idempotency_key = $1::text LIMIT 1;",
"options": {
"queryReplacement": "={{ $json.ctx.idempotency_key }}"
}
},
"name": "Check Idempotency",
"type": "n8n-nodes-base.postgres",
"typeVersion": 2.6,
"position": [500, 0],
"credentials": {
```

```
"postgres": {
"id": "SFNQsmuu4zirZAnP",
"name": "Postgres account"
}
},
"alwaysOutputData": true,
"onError": "continueErrorOutput"
},
{
"parameters": {
"jsCode": "const items = $input.all();\nconst ctx = $('Validate
Input').first().json.ctx;\nconst dup = (items.length > 0 && items[0].json.booking_id) ?
items[0].json : null;\nif (dup) {\n  return [{ json: { success: true, error_code: null,
error_message: null, data: { booking_id: dup.booking_id, status: dup.status, gcal_event_id:
dup.gcal_event_id, is_duplicate: true }, _meta: ctx._meta }}];\n}\nreturn [{ json: { ctx,
is_duplicate: false } }];"
},
"name": "Process Idempotency",
"type": "n8n-nodes-base.code",
"typeVersion": 2,
"position": [750, 0]
},
{
"parameters": {
"conditions": {
"options": {
"caseSensitive": true,
"leftValue": "",
"typeValidation": "strict"
},
"conditions": [
{
"id": "dup",
"leftValue": "={{ $json.is_duplicate }}",
"rightValue": true,
"operator": {
"type": "boolean",
"operation": "equals"
}
}
],
"combinator": "and"
}
},
"name": "Is Duplicate?",
"type": "n8n-nodes-base.if",
"typeVersion": 2.3,
```

```
"position": [1000, 0]
},
{
"parameters": {
"jsCode": "const ctx = $input.first().json.ctx;\nconst lock_key =
`lock_${ctx.provider_id}_${ctx.start_time}`;\nconst owner_token =
ctx.idempotency_key;\nreturn [{ json: { ctx, lock_key, owner_token, lock_query: `INSERT INTO
booking_locks (provider_id, start_time, lock_key, owner_token, expires_at, created_at) VALUES
(${ctx.provider_id}, '${ctx.start_time}'::timestamptz, '${lock_key}', '${owner_token}', NOW()
+ INTERVAL '5 minutes', NOW()) ON CONFLICT (lock_key) DO UPDATE SET expires_at = NOW() +
```

INTERVAL '5 minutes', updated_at = NOW() WHERE booking_locks.expires_at < NOW() RETURNING id, owner_token;` }}];"
},
"name": "Build Lock Query",
"type": "n8n-nodes-base.code",
"typeVersion": 2,
"position": [1250, -150]
},
{
"parameters": {
"operation": "executeQuery",
"query": "={{ $json.lock_query }}",
"options": {}
},
"name": "Acquire Lock",
"type": "n8n-nodes-base.postgres",
"typeVersion": 2.6,
"position": [1500, -150],
"credentials": {
"postgres": {
"id": "SFNQsmuu4zirZAnP",
"name": "Postgres account"
}
},
"alwaysOutputData": true,
"onError": "continueErrorOutput"
},
{
"parameters": {
"jsCode": "const items = $input.all();\nconst ctx = $('Build Lock Query').first().json.ctx;\nconst lock_key = $('Build Lock Query').first().json.lock_key;\nconst owner_token = $('Build Lock Query').first().json.owner_token;\nconst lock_acquired = (items.length > 0 && items[0].json.id) ? true : false;\nif (!lock_acquired) {\n return [{ json: { success: false, error_code: 'LOCK_DENIED', error_message: 'Slot already booked', data: { lock_key }, _meta: ctx._meta, _needs_lock_release: false } }];\n}\nreturn [{ json: { ctx, lock_key, owner_token, lock_acquired: true } }];"
},
"name": "Process Lock",
"type": "n8n-nodes-base.code",
"typeVersion": 2,
"position": [1750, -150]
},
{
"parameters": {
"conditions": {
"options": {
"caseSensitive": true,
"leftValue": "",
"typeValidation": "strict"
},
"conditions": [
{
"id": "lock",
"leftValue": "={{ $json.lock_acquired }}",
"rightValue": true,
"operator": {
"type": "boolean",

```
"operation": "equals"
}
}
],
"combinator": "and"
}
},
"name": "Lock Acquired?",
"type": "n8n-nodes-base.if",
"typeVersion": 2.3,
"position": [2000, -150]
},
{
"parameters": {
"jsCode": "const ctx = $input.first().json.ctx;\nconst lock_key =
$input.first().json.lock_key;\nconst owner_token = $input.first().json.owner_token;\nreturn
[{ json: { ctx, lock_key, owner_token, cb_query: `SELECT allowed, failure_count FROM
circuit_breaker WHERE service_id = 'google_calendar' LIMIT 1;` }}];"
},
"name": "Build CB Query",
"type": "n8n-nodes-base.code",
"typeVersion": 2,
"position": [2250, -250]
},
{
"parameters": {
"operation": "executeQuery",
"query": "={{ $json.cb_query }}",
"options": {}
```

```
},
"name": "Check Circuit Breaker",
"type": "n8n-nodes-base.postgres",
"typeVersion": 2.6,
"position": [2500, -250],
"credentials": {
"postgres": {
"id": "SFNQsmuu4zirZAnP",
"name": "Postgres account"
}
},
"alwaysOutputData": true,
"onError": "continueErrorOutput"
},
{
"parameters": {
"jsCode": "const items = $input.all();\nconst ctx = $('Build CB
Query').first().json.ctx;\nconst lock_key = $('Build CB Query').first().json.lock_key;\nconst
owner_token = $('Build CB Query').first().json.owner_token;\nconst cb_row = (items.length > 0
&& items[0].json) ? items[0].json : null;\nconst cb_allowed = cb_row && cb_row.allowed ===
true;\nif (!cb_allowed) {\n  return [{ json: { success: false, error_code:
'CIRCUIT_BREAKER_OPEN', error_message: 'GCal service unavailable', data: { service:
'google_calendar', failure_count: cb_row?.failure_count || 0 }, _meta: ctx._meta,
_needs_lock_release: true, lock_key, owner_token } }];\n}\nreturn [{ json: { ctx, lock_key,
owner_token, cb_allowed: true } }];"
},
"name": "Process Circuit Breaker",
"type": "n8n-nodes-base.code",
```

"typeVersion": 2,
"position": [2750, -250]
},
{
"parameters": {
"conditions": {
"options": {
"caseSensitive": true,
"leftValue": "",
"typeValidation": "strict"
},
"conditions": [
{
"id": "cb",
"leftValue": "={{ $json.cb_allowed }}",
"rightValue": true,
"operator": {
"type": "boolean",
"operation": "equals"
}
}
],
"combinator": "and"
}
},
"name": "CB Allowed?",
"type": "n8n-nodes-base.if",
"typeVersion": 2.3,
"position": [3000, -250]
},
{
"parameters": {
"jsCode": "const ctx = $input.first().json.ctx;\nconst lock_key =
$input.first().json.lock_key;\nconst owner_token = $input.first().json.owner_token;\nreturn
[{ json: { ctx, lock_key, owner_token, avail_query: `SELECT COUNT(*) as count FROM bookings
WHERE provider_id = ${ctx.provider_id} AND service_id = ${ctx.service_id} AND start_time =
'${ctx.start_time}'::timestamptz AND status = 'CONFIRMED';` }}];"
},
"name": "Build Avail Query",
"type": "n8n-nodes-base.code",
"typeVersion": 2,
"position": [3250, -350]
},
{
"parameters": {
"operation": "executeQuery",
"query": "={{ $json.avail_query }}",
"options": {}
},
"name": "Check Availability",
"type": "n8n-nodes-base.postgres",
"typeVersion": 2.6,
"position": [3500, -350],
"credentials": {
"postgres": {
"id": "SFNQsmuu4zirZAnP",
"name": "Postgres account"
}

```
},
"alwaysOutputData": true,
"onError": "continueErrorOutput"
},
{
"parameters": {
```

```
"jsCode": "const items = $input.all();\nconst ctx = $('Build Avail
Query').first().json.ctx;\nconst lock_key = $('Build Avail
Query').first().json.lock_key;\nconst owner_token = $('Build Avail
Query').first().json.owner_token;\nconst avail_row = (items.length > 0 && items[0].json) ?
items[0].json : null;\nconst avail_count = parseInt(avail_row?.count || '0', 10);\nconst
is_available = avail_count === 0;\nif (!is_available) {\n  return [{ json: { success: false,
error_code: 'NO_AVAILABILITY', error_message: 'Slot not available', data: { provider_id:
ctx.provider_id, service_id: ctx.service_id, start_time: ctx.start_time }, _meta: ctx._meta,
_needs_lock_release: true, lock_key, owner_token } }];\n}\nreturn [{ json: { ctx, lock_key,
owner_token, is_available: true } }];"
},
"name": "Process Availability",
"type": "n8n-nodes-base.code",
"typeVersion": 2,
"position": [3750, -350]
},
{
"parameters": {
"conditions": {
"options": {
"caseSensitive": true,
"leftValue": "",
"typeValidation": "strict"
},
"conditions": [
{
"id": "avail",
"leftValue": "={{ $json.is_available }}",
"rightValue": true,
"operator": {
"type": "boolean",
"operation": "equals"
}
}
],
"combinator": "and"
}
},
"name": "Is Available?",
"type": "n8n-nodes-base.if",
"typeVersion": 2.3,
"position": [4000, -350]
},
{
"parameters": {
"calendar": {
"__rl": true,
"value": "dev.n8n.stax@gmail.com",
"mode": "list",
"cachedResultName": "dev.n8n.stax@gmail.com"
},
```

```
"event": {
"start": "={{ $json.ctx.start_time }}",
"end": "={{ $json.ctx.end_time }}",
"summary": "={{ $json.ctx.event_title }}",
"description": "={{ `Booking: ${$json.ctx.customer_id || $json.ctx.chat_id}\\nProvider:
${$json.ctx.provider_id}\\nService: ${$json.ctx.service_id}` }}"
}
},
"name": "Create GCal Event",
"type": "n8n-nodes-base.googleCalendar",
"typeVersion": 1.3,
"position": [4250, -450],
"credentials": {
"googleCalendarOAuth2Api": {
"id": "OsRBfz3Cs7Ph5uV5",
"name": "Google Calendar account"
}
},
"onError": "continueErrorOutput"
},
{
"parameters": {
"jsCode": "const items = $input.all();\nconst ctx = $('Create GCal
Event').first().json.ctx;\nconst lock_key = $('Create GCal
Event').first().json.lock_key;\nconst owner_token = $('Create GCal
Event').first().json.owner_token;\nconst gcal_row = (items.length > 0 && items[0].json) ?
items[0].json : null;\nconst gcal_event_id = gcal_row?.id || null;\nconst gcal_success =
!!gcal_event_id;\nif (!gcal_success) {\n  return [{ json: { success: false, error_code:
'GCAL_ERROR', error_message: gcal_row?.message || 'Failed to create GCal event', data: {
service: 'google_calendar' }, _meta: ctx._meta, _needs_lock_release: true, lock_key,
owner_token } }];\n}\nreturn [{ json: { ctx, lock_key, owner_token, gcal_event_id,
gcal_success: true } }];"
},
"name": "Process GCal",
"type": "n8n-nodes-base.code",
"typeVersion": 2,
"position": [4500, -450]
},
{
"parameters": {
"conditions": {
"options": {
"caseSensitive": true,
"leftValue": "",
"typeValidation": "strict"
},
"conditions": [
{
```

```
"id": "gcal",
"leftValue": "={{ $json.gcal_success }}",
"rightValue": true,
"operator": {
"type": "boolean",
"operation": "equals"
}
}
],
```

"combinator": "and"
}
},
"name": "GCal Success?",
"type": "n8n-nodes-base.if",
"typeVersion": 2.3,
"position": [4750, -450]
},
{
"parameters": {
"operation": "executeQuery",
"query": "INSERT INTO bookings (provider_id, service_id, start_time, end_time, idempotency_key, gcal_event_id, user_id, status, created_at) VALUES ($1::int, $2::int, $3::timestamp, $4::timestamp, $5::text, $6::text, $7::bigint, 'CONFIRMED', NOW()) RETURNING id;",
"options": {
"queryReplacement": "={{ [$json.ctx.provider_id, $json.ctx.service_id, $json.ctx.start_time, $json.ctx.end_time, $json.ctx.idempotency_key, $json.gcal_event_id, $json.ctx.user_id] }}"
}
},
"name": "Create Booking",
"type": "n8n-nodes-base.postgres",
"typeVersion": 2.6,
"position": [5000, -550],
"credentials": {
"postgres": {
"id": "SFNQsmuu4zirZAnP",
"name": "Postgres account"
}
},
"alwaysOutputData": true,
"onError": "continueErrorOutput"
},
{
"parameters": {
"jsCode": "const items = $input.all();\nconst ctx = $('Create Booking').first().json.ctx;\nconst lock_key = $('Create Booking').first().json.lock_key;\nconst owner_token = $('Create Booking').first().json.owner_token;\nconst gcal_event_id = $('Create Booking').first().json.gcal_event_id;\nconst booking_row = (items.length > 0 && items[0].json) ? items[0].json : null;\nconst booking_id = booking_row?.id || null;\nconst booking_success = !!booking_id;\nif (!booking_success) {\n return [{ json: { success: false, error_code: 'DB_ERROR', error_message: 'Failed to create booking', data: { gcal_event_id, needs_rollback: true, needs_lock_release: true }, _meta: ctx._meta, lock_key, owner_token } }];\n}\nreturn [{ json: { ctx, lock_key, owner_token, gcal_event_id, booking_id, booking_success: true } }];"
},
"name": "Process Booking",
"type": "n8n-nodes-base.code",
"typeVersion": 2,
"position": [5250, -550]
},
{
"parameters": {
"conditions": {
"options": {
"caseSensitive": true,
"leftValue": "",

```
"typeValidation": "strict"
},
"conditions": [
{
"id": "booking",
"leftValue": "={{ $json.booking_success }}",
"rightValue": true,
"operator": {
"type": "boolean",
"operation": "equals"
}
}
],
"combinator": "and"
}
},
"name": "Booking Success?",
"type": "n8n-nodes-base.if",
"typeVersion": 2.3,
"position": [5500, -550]
},
{
"parameters": {
"operation": "executeQuery",
"query": "DELETE FROM booking_locks WHERE lock_key = $1::text;",
"options": {
"queryReplacement": "={{ [$json.lock_key] }}"
```

```
}
},
"name": "Release Lock Success",
"type": "n8n-nodes-base.postgres",
"typeVersion": 2.6,
"position": [5750, -550],
"credentials": {
"postgres": {
"id": "SFNQsmuu4zirZAnP",
"name": "Postgres account"
}
},
"onError": "continueErrorOutput"
},
{
"parameters": {
"jsCode": "const ctx = $('Process Booking').first().json.ctx;\nconst booking_id = $('Process
Booking').first().json.booking_id;\nconst gcal_event_id = $('Process
Booking').first().json.gcal_event_id;\nreturn [{ json: { success: true, error_code: null,
error_message: null, data: { booking_id, idempotency_key: ctx.idempotency_key, provider_id:
ctx.provider_id, service_id: ctx.service_id, start_time: ctx.start_time, end_time:
ctx.end_time, status: 'CONFIRMED', gcal_event_id, is_duplicate: false }, _meta: { source:
'WF2_Booking_Orchestrator_v4', timestamp: new Date().toISOString(), version: '4.0.0-internal'
} }}];"
},
"name": "Success Output",
"type": "n8n-nodes-base.code",
"typeVersion": 2,
"position": [6000, -550]
},
```

{
"parameters": {
"operation": "executeQuery",
"query": "DELETE FROM booking_locks WHERE lock_key = $1::text;",
"options": {
"queryReplacement": "={{ [$json.lock_key] }}"
}
},
"name": "Release Lock Error",
"type": "n8n-nodes-base.postgres",
"typeVersion": 2.6,
"position": [3500, -150],
"credentials": {
"postgres": {
"id": "SFNQsmuu4zirZAnP",
"name": "Postgres account"
}
},
"onError": "continueErrorOutput"
},
{
"parameters": {
"jsCode": "const input = $input.first().json;\nconst error_response = { success: input.success || false, error_code: input.error_code || 'UNKNOWN_ERROR', error_message: input.error_message || 'An error occurred', data: input.data || null, _meta: input._meta || { source: 'WF2_v4', timestamp: new Date().toISOString() } };\nif (input._needs_lock_release && input.lock_key) {\n return [{ json: { ...error_response, _release_lock: true, lock_key: input.lock_key, owner_token: input.owner_token } }];\n}\nreturn [{ json: error_response }];"
},
"name": "Error Output",
"type": "n8n-nodes-base.code",
"typeVersion": 2,
"position": [3750, -150]
}
],
"connections": {
"Webhook": {
"main": [[{"node": "Validate Input", "type": "main", "index": 0}]]
},
"Validate Input": {
"main": [[{"node": "Check Idempotency", "type": "main", "index": 0}]]
},
"Check Idempotency": {
"main": [[{"node": "Process Idempotency", "type": "main", "index": 0}]]
},
"Process Idempotency": {
"main": [[{"node": "Is Duplicate?", "type": "main", "index": 0}]]
},
"Is Duplicate?": {
"main": [
[[{"node": "Error Output", "type": "main", "index": 0}]],
[[{"node": "Build Lock Query", "type": "main", "index": 0}]]
]
},
"Build Lock Query": {
"main": [[{"node": "Acquire Lock", "type": "main", "index": 0}]]
},
"Acquire Lock": {

```
"main": [[{"node": "Process Lock", "type": "main", "index": 0}]]
},
"Process Lock": {
"main": [[{"node": "Lock Acquired?", "type": "main", "index": 0}]]
```

```
},
"Lock Acquired?": {
"main": [
[[{"node": "Error Output", "type": "main", "index": 0}]],
[[{"node": "Build CB Query", "type": "main", "index": 0}]]
]
},
"Build CB Query": {
"main": [[{"node": "Check Circuit Breaker", "type": "main", "index": 0}]]
},
"Check Circuit Breaker": {
"main": [[{"node": "Process Circuit Breaker", "type": "main", "index": 0}]]
},
"Process Circuit Breaker": {
"main": [[{"node": "CB Allowed?", "type": "main", "index": 0}]]
},
"CB Allowed?": {
"main": [
[[{"node": "Error Output", "type": "main", "index": 0}]],
[[{"node": "Build Avail Query", "type": "main", "index": 0}]]
]
},
"Build Avail Query": {
"main": [[{"node": "Check Availability", "type": "main", "index": 0}]]
},
"Check Availability": {
"main": [[{"node": "Process Availability", "type": "main", "index": 0}]]
},
"Process Availability": {
"main": [[{"node": "Is Available?", "type": "main", "index": 0}]]
},
"Is Available?": {
"main": [
[[{"node": "Error Output", "type": "main", "index": 0}]],
[[{"node": "Create GCal Event", "type": "main", "index": 0}]]
]
},
"Create GCal Event": {
"main": [[{"node": "Process GCal", "type": "main", "index": 0}]]
},
"Process GCal": {
"main": [[{"node": "GCal Success?", "type": "main", "index": 0}]]
},
"GCal Success?": {
"main": [
[[{"node": "Error Output", "type": "main", "index": 0}]],
[[{"node": "Create Booking", "type": "main", "index": 0}]]
]
},
"Create Booking": {
"main": [[{"node": "Process Booking", "type": "main", "index": 0}]]
},
"Process Booking": {
```

"main": [[{"node": "Booking Success?", "type": "main", "index": 0}]]
},
"Booking Success?": {
"main": [
[[{"node": "Error Output", "type": "main", "index": 0}]],
[[{"node": "Release Lock Success", "type": "main", "index": 0}]]
]
},
"Release Lock Success": {
"main": [[{"node": "Success Output", "type": "main", "index": 0}]]
},
"Error Output": {
"main": [[{"node": "Release Lock Error", "type": "main", "index": 0}]]
},
"Release Lock Error": {
"main": [[]]
}
},
"settings": {
"executionOrder": "v1",
"saveDataErrorExecution": "all",
"saveDataSuccessExecution": "all",
"saveManualExecutions": true,
"saveExecutionProgress": true
},
"staticData": null,
"tags": [

```json
{
"name": "production"
},
{
"name": "booking"
},
{
"name": "v4"
}
],
"triggerCount": 1,
"updatedAt": "2026-03-17T00:00:00.000Z",
"versionId": "v4.0.0"
}
```